
WMAttack: Automated Attack Search for Adversarial Evaluation of World-Model Agents

Zhixiang Guo

Nanyang Technological University
zhixiang004@e.ntu.edu.sg

Siyuan Liang

Nanyang Technological University
siyuan.liang@ntu.edu.sg

Shi Fu

Nanyang Technological University
SHI011@e.ntu.edu.sg

Cheng Guo

Nanyang Technological University
guo007@e.ntu.edu.sg

András Balogh

University of Szeged
abalogh@inf.u-szeged.hu

Márk Jelasity

University of Szeged
jelasity@inf.u-szeged.hu

Dacheng Tao

Nanyang Technological University
dacheng.tao@ntu.edu.sg

Abstract

Despite the growing use of world models as decision-making agents, their adversarial robustness remains underexplored due to the lack of dedicated automated evaluation methods. A key obstacle is that attack evaluation must be both accurate and efficient: weak manually tuned attacks can overestimate robustness, while exhaustive hyperparameter search is prohibitively expensive because each candidate requires closed-loop rollouts through learned latent dynamics. We introduce **WMAttack**, an automated attack-search framework for adversarial evaluation of world-model agents. WMAttack formulates robustness evaluation as a finite-budget search over attack configurations, including attack families, perturbation budgets, optimization steps, restarts, and allocation rules. To improve search accuracy, **Self-Correcting Attack Search** (SCAS) refines the attack proposal distribution using feedback from reward degradation, action instability, runtime cost, and rollout variability. To improve search efficiency, **Representation-Guided Attack Retrieval** (RGAR) retrieves effective historical configurations from representation-similar tasks, providing a warm start for unseen environments. We provide a theoretical explanation showing that proposal refinement improves finite-budget search when it shifts probability mass toward high-utility attacks. Across Atari and DeepMind Control tasks, WMAttack consistently discovers stronger attacks than the evaluated baselines, improving normalized reward drop from 0.497 to 1.034 on DreamerV3 Atari and from 0.319 to 0.682 on DMC. Ablations further show that RGAR improves initial candidate quality and SCAS improves final attack utility under fixed evaluation budgets.

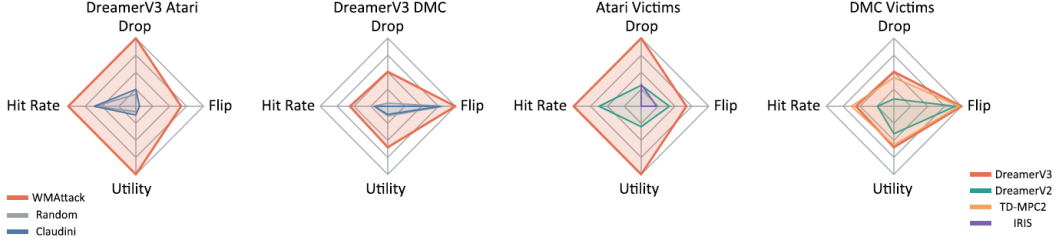


Figure 1: Overview of WMAAttack effectiveness across tasks and world-model agents. Metrics are min-max normalized for visualization only; larger radius indicates stronger attack or search outcome, with exact values reported in Tables 1 and 2.

1 Introduction

World models have emerged as a foundational paradigm for decision-making agents, enabling sample-efficient planning and prediction through learned latent dynamics [18, 19, 22]. Their growing use has also motivated safety-oriented world-model methods that incorporate constraints, uncertainty, or Lyapunov-style critics into model-based reinforcement learning [26, 3, 54]. However, superior performance on visual control [42], Atari [6], and continuous-control benchmarks [23] does not guarantee robustness. Recent work on robust latent representations, latent safety filters, and surprise-based rejection further suggests that failures often arise from representation shift and out-of-distribution transitions [41, 40, 58]. Small, budget-constrained perturbations can propagate through learned latent representations and predictive dynamics [37, 16, 17], latent transition models [57], policies [55], or adversarially corrupted planning objectives and trajectory rankings [52, 27, 39, 11], leading to unstable decisions or severe performance degradation. Despite these risks, we still lack a dedicated automated attack-search framework [34] to systematically evaluate world-model agents across diverse tasks, models, and attack families. This raises a critical question: how can we efficiently expose and quantify the failure modes of world-model agents under adversarial conditions?

Evaluating robustness in this context is challenging because attack efficacy is highly sensitive to both environment dynamics and adversarial hyperparameters [51, 24, 38]. World-model environments differ in observation spaces, reward scales, temporal structures, and policy sensitivities [23, 45], while robustness itself involves return degradation, action instability, and latent representation drift [55, 44, 4]. Manual tuning is therefore unreliable and inefficient: weak attacks may create pseudo-robustness, while exhaustive search is computationally prohibitive because each candidate may require closed-loop rollouts and repeated latent-dynamics computation [8, 10, 30]. An effective evaluation method must balance accuracy and efficiency by avoiding cold-start search and adaptively refining configurations from agent feedback. These challenges motivate an automated framework that leverages cross-task experience and concentrates limited evaluation budgets on high-impact adversarial regions.

In this work, we propose **WMAAttack**, an automated attack-search framework for adversarial evaluation of world-model agents. Rather than relying on fixed attacks or heuristic manual tuning, WMAAttack formulates robustness evaluation as a finite-budget search problem over adversarial configurations, including perturbation budgets, optimization steps, restarts, schedules, and allocation rules. To navigate this large search space, WMAAttack employs a two-stage optimization process. First, **Representation-Guided Attack Retrieval (RGAR)** exploits intrinsic similarities between world models by projecting diverse dynamics into a unified latent representation space and retrieving effective historical attack configurations as a warm start for new tasks. This enables cross-task experience transfer and improves initialization over zero-knowledge starting points. Second, **Self-Correcting Attack Search (SCAS)** iteratively refines the attack strategy by using multi-dimensional feedback, such as reward degradation and action instability, as corrective signals to update a proposal distribution over the attack parameter space. By shifting probability mass toward regions of higher adversarial efficacy, WMAAttack concentrates its search on potent configurations, reduces redundant trials, and provides a theoretical basis for discovering stronger attacks under a fixed evaluation budget.

Our contributions are summarized as follows:

- We formulate adversarial robustness evaluation for world models as a finite-budget attack-configuration search problem, where the goal is to efficiently identify high-impact attacks over diverse tasks, models, attack methods, and hyperparameter settings.
- We develop **WMAttack**, an automated attack-search framework that combines **Representation-Guided Attack Retrieval (RGAR)** with **Self-Correcting Attack Search (SCAS)**, and provide a theoretical explanation for why feedback-driven proposal refinement can improve attack-search efficiency.
- We evaluate WMAttack on 46 DreamerV3 Atari and DeepMind Control tasks, where it discovers stronger attacks than random search and a Claudini-style baseline, and further report cross-model evaluations on DreamerV2, TD-MPC2, and IRIS, showing that it discovers stronger attacks than random search and a Claudini-style autoresearch baseline, with consistent gains in reward drop, action instability, and scalarized attack utility.

2 Related Work

2.1 World Models in Reinforcement Learning

World models are a central framework in model-based reinforcement learning, enabling agents to learn predictive latent dynamics for planning and decision making. Unlike vision-language-action models that often map multimodal inputs directly to actions through foundation models [7], RL-based world models emphasize environment dynamics, imagination, and sample-efficient control. Early methods such as PlaNet [19] and Dreamer [20] introduced recurrent state-space models for learning compact latent representations from pixels. Subsequent systems, including DreamerV2 and DreamerV3, scaled latent imagination to Atari, continuous control, and diverse RL domains [21, 22]. More recent architectures replace or augment recurrent dynamics with Transformer-based sequence modeling, as in IRIS and STORM [37, 57], or generative transition models, as in DIAMOND [1]. Our work focuses on this class of RL-based world-model agents because their decisions are mediated by learned latent dynamics: perturbations to observations can propagate through the encoder, transition model, and policy, making closed-loop adversarial evaluation especially important [56, 46, 28].

2.2 Adversarial Robustness and Evaluation

Adversarial examples show that small, bounded perturbations can cause severe failures in neural models [15, 49, 32]. In reinforcement learning, the problem is more complex because perturbations affect actions, future states, and long-horizon returns. Prior work formalizes observation attacks through the State-Adversarial MDP framework [55] and shows that policies are vulnerable to observation perturbations, strategically timed attacks, and adversarial policies [25, 33, 14]. For world-model agents, these risks are amplified by learned latent dynamics [50]: corrupted observations may distort latent states and imagined trajectories, leading to compounding errors in downstream control.

Robustness evaluation for world models remains fragmented. Recent diagnostic benchmarks such as WorldBench and iWorld-Bench study physical reasoning, interaction, or causal consistency in predictive models [44, 13], but they do not provide an automated attack-search framework for closed-loop RL world-model agents. Meanwhile, static adversarial evaluation protocols such as AutoAttack [8, 31, 29] reduce manual tuning for supervised vision models, but do not address the high-dimensional attack-configuration space and rollout cost of world-model evaluation. Recent autoresearch-style systems such as Claudini [38, 47] show that feedback-driven agents can discover strong adversarial attacks for LLMs, but their setting differs from closed-loop, dynamics-driven world models. WMAttack bridges this gap by treating adversarial evaluation as automated finite-budget attack search, using cross-task retrieval and feedback-driven refinement to identify high-impact configurations for world models.

3 Preliminaries

3.1 Victim and Threat Model

We consider adversarial evaluation of a trained world-model agent under closed-loop interaction with an environment. The victim agent is fixed during evaluation and is denoted by

$$M_\theta = (\phi_\theta, f_\theta, \pi_\theta), \quad (1)$$

where ϕ_θ is the encoder or representation module, f_θ is the latent dynamics model, and π_θ is the policy. At each time step, the agent maps observations into latent states and selects actions from these internal representations. This abstraction covers recurrent state-space world models [21, 22], Transformer-based world models [37], and latent dynamics models [23].

The attack is applied only at evaluation time. The attacker does not modify the environment dynamics, reward function, or world-model parameters. Instead, adversarial influence enters the closed-loop trajectory only through perturbed observations received by the agent. Thus, a perturbation can affect long-horizon behavior only by propagating through the encoder, latent dynamics, and policy. Additional details on the clean and attacked closed-loop processes are provided in Appendix A.

3.2 Adversary Capabilities

For an evaluation task τ , let $\mathcal{C}_\tau$ denote the attack configuration space. An attack configuration $c \in \mathcal{C}_\tau$ specifies both the attack family and its evaluation hyperparameters:

$$c = (\text{attack type}, \epsilon, \text{steps}, \text{restarts}, \rho, \text{seed}, \text{allocation rule}), \quad (2)$$

where ϵ is the perturbation budget, steps and restarts control attack optimization, ρ controls the adaptive step-size schedule when applicable, and the allocation rule specifies how attack resources are assigned during evaluation. In our implementation, the attack family is selected from APGD-CE and APGD-DLR [8], FAB [9], Square Attack [2], and Physics-Conditioned World-Model Attack (PhysCond-WMA) [17]. Detailed hyperparameter ranges are given in Appendix B.

We consider bounded observation-space attacks on the image keys used by the world-model encoder. Let $\mathcal{K}_{\text{img}}$ denote these keys. For each $k \in \mathcal{K}_{\text{img}}$, the attacker replaces the clean observation with

$$\tilde{o}_{t,k} = \Pi_{[-0.5, 0.5]}(o_{t,k} + \delta_{t,k}), \quad \|\delta_{t,k}\|_\infty \leq \epsilon/255, \quad (3)$$

where $\Pi_{[-0.5, 0.5]}$ projects the perturbed observation back to the normalized input range. Different attack families may use different optimization procedures or access assumptions, but WMAAttack evaluates all of them under the same configuration space and utility function.

3.3 Attack Goal

The goal of WMAAttack is not to certify robustness or guarantee a global optimum, but to efficiently discover high-impact attacks under a finite evaluation budget. Let $J_{\text{clean}}(\tau)$ denote the expected episodic return under clean observations, and let $J_{\text{adv}}(\tau, c)$ denote the attacked return under configuration c . WMAAttack scores each configuration by a scalarized utility:

$$U_\tau(c) = D_\tau(c) + w_f F_\tau(c) - w_r \log(1 + T_\tau(c)) - w_v V_\tau(c), \quad (4)$$

where $D_\tau(c)$ is normalized reward degradation, $F_\tau(c)$ measures action instability, $T_\tau(c)$ is evaluation time, $V_\tau(c)$ measures rollout variability, and w_f, w_r, w_v balance these terms. The exact definitions of the utility components are provided in Appendix C.

Under a finite budget B , adversarial evaluation is formulated as attack-configuration search:

$$\max_{c_1, \dots, c_B \in \mathcal{C}_\tau} \mathbb{E} \left[\max_{1 \leq i \leq B} U_\tau(c_i) \right]. \quad (5)$$

This formulation reflects the practical cost of world-model evaluation, where each candidate configuration may require closed-loop rollouts and repeated latent-dynamics computation.

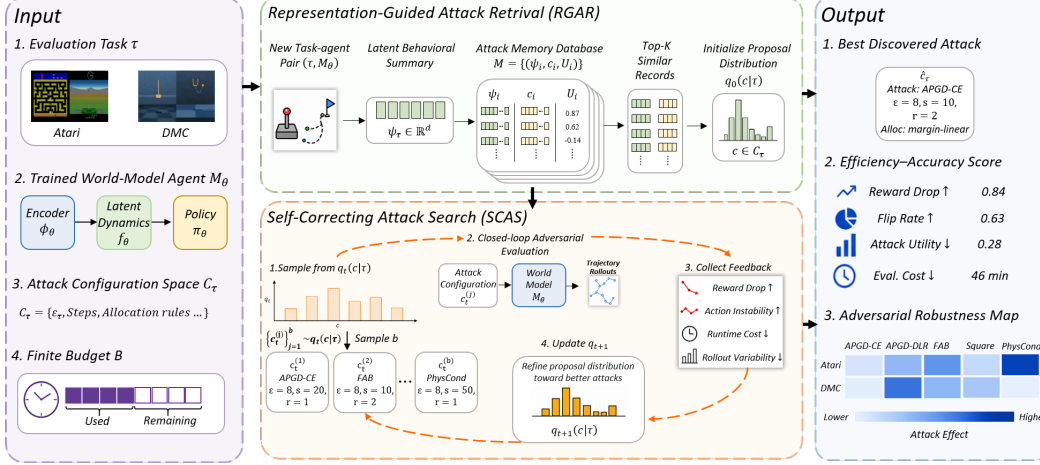


Figure 2: Overview of WMAttack. RGAR initializes the attack proposal distribution from representation-similar historical tasks, and SCAS refines it using closed-loop rollout feedback.

4 Method

4.1 Problem Decomposition

WMAttack takes the finite-budget attack-search objective in Section 3.3 as its starting point and learns how to allocate the limited evaluation budget across C_τ . Instead of sampling configurations independently from a fixed prior, WMAttack maintains an adaptive proposal distribution $q_t(c|\tau)$ over attack configurations. This distribution represents the current belief about which configurations are likely to produce high utility on task τ .

At the beginning of search, the main challenge is cold-start initialization: for a new task-agent pair, the evaluator does not know which attack family or hyperparameter region is likely to be effective. WMAttack addresses this by using **Representation-Guided Attack Retrieval** (RGAR) to initialize $q_0(c|\tau)$ from historical configurations that were effective on representation-similar tasks. After each evaluation round, the challenge becomes adaptation: the method must use trial outcomes to refine the search direction. WMAttack addresses this with **Self-Correcting Attack Search** (SCAS), which updates $q_t(c|\tau)$ using feedback from reward degradation, action instability, evaluation cost, and rollout variability. Thus, the finite-budget search problem is decomposed into two coupled steps:

$$\text{RGAR: } \mathcal{M}, \psi_\tau \mapsto q_0(c|\tau), \quad \text{SCAS: } q_t(c|\tau), \mathcal{H}_t \mapsto q_{t+1}(c|\tau), \quad (6)$$

where $\mathcal{M}$ is the attack memory, ψ_τ is the latent behavioral summary of the new task-agent pair, and $\mathcal{H}_t$ is the accumulated evaluation history. The next two sections describe these two steps in detail.

4.2 Representation-Guided Attack Retrieval

A central challenge in attacking world-model agents is that attack effectiveness varies substantially across tasks, models, and environments. However, agents with similar latent dynamics or decision behaviors may share effective attack configurations [5, 53, 48]. Tasks with similar temporal dependencies, reward sensitivities, or latent transition patterns may respond similarly to perturbation budgets, optimization steps, or allocation rules. RGAR exploits this observation by retrieving prior adversarial experience through latent behavioral representations [35, 36].

For each task-agent pair, WMAttack constructs a latent behavioral summary

$$\psi_\tau = g(\{z_t, \hat{z}_{t+k}, u_t, r_t\}_{t=1}^H), \quad (7)$$

where z_t denotes the agent's latent state, $\hat{z}_{t+k}$ denotes predicted future latent states, u_t is the selected action, and r_t is the observed reward. The function $g(\cdot)$ aggregates trajectory-level information into a compact representation that characterizes the behavior of the agent under clean evaluation.

Given a new task summary ψ_τ , RGAR retrieves the top- K most relevant historical records:

$$\mathcal{N}_K(\tau) = \text{TopK}_i \text{sim}(\psi_\tau, \psi_i), \quad (8)$$

where $\text{sim}(\cdot, \cdot)$ measures similarity in the latent representation space. The retrieved configurations are then converted into a warm-start proposal distribution:

$$q_0(c \mid \tau) = (1 - \lambda)q_{\text{base}}(c) + \lambda \sum_{i \in \mathcal{N}_K(\tau)} \alpha_i \delta(c = c_i), \quad (9)$$

where q_{base} is a generic prior over attack configurations, $\delta(\cdot)$ is a point mass on a retrieved configuration, and $\lambda \in [0, 1]$ controls the strength of retrieval. The weights α_i are assigned according to retrieval similarity and historical attack utility. This retrieval stage reduces cold-start search. Rather than initializing from uninformed random configurations, WMAttack starts from configurations that were effective for representation-similar task-agent pairs, improving the initial attack proposal distribution before task-specific feedback is collected.

4.3 Self-Correcting Attack Search

RGAR provides an informed warm start, but the retrieved configurations may not be optimal for a new task. SCAS therefore refines the search process using feedback from evaluated attack trials. We view the ranked candidate generator as an implicit proposal distribution $q_t(c \mid \tau)$ over the attack configuration space. At round t , WMAttack proposes a batch of candidate configurations and evaluates them through closed-loop rollouts. Each candidate is scored by the scalarized utility $U_\tau(c)$ defined in Section 3.3. In implementation, WMAttack uses a scout-confirm protocol: inexpensive scout rollouts estimate candidate utility, and the most promising candidates are re-evaluated with more episodes for confirmation. The search history is updated as

$$\mathcal{H}_t = \mathcal{H}_{t-1} \cup \{(c_t^{(j)}, U_\tau(c_t^{(j)}), A_t^{(j)})\}_{j=1}^b, \quad (10)$$

where $A_t^{(j)}$ denotes the feedback signal for candidate $c_t^{(j)}$, including observed failure patterns, parameter recommendations, and search-direction suggestions.

SCAS uses this feedback to generate and rank new candidates. High-utility trials serve as anchors for local refinement, while feedback signals adjust the search direction by changing ϵ , optimization steps, restarts, or the allocation rule. Abstractly, the proposal update is written as

$$q_{t+1}(c \mid \tau) = (1 - \alpha_t)q_t(c \mid \tau) + \alpha_t \hat{q}_t(c \mid \mathcal{H}_t), \quad (11)$$

where α_t is the update rate and $\hat{q}_t$ is the feedback-induced proposal distribution estimated from the accumulated search history. In implementation, this update is realized through candidate generation, local neighborhood expansion, feedback-guided variants, and ranked selection rather than explicit probabilistic sampling. This self-correcting loop concentrates the evaluation budget on regions of the attack space that are more likely to expose meaningful world-model failures.

4.4 Theoretical Analysis

We provide a concise explanation for why RGAR and SCAS can improve finite-budget attack search. The full derivation, including the construction of the utility-biased reference distribution, noisy correction, comparison with a Claudini-style baseline, and finite-episode utility estimation, is deferred to Appendix D.

Let $\mathcal{G}_\eta$ be the set of η -effective attack configurations:

$$\mathcal{G}_\eta = \{c \in \mathcal{C}_\tau : U_\tau(c) \geq U_\tau^* - \eta\}, \quad U_\tau^* = \max_{c \in \mathcal{C}_\tau} U_\tau(c). \quad (12)$$

At round t , let q_t be the implicit proposal distribution induced by retrieval context, candidate ranking, and search feedback, and define $p_t = q_t(\mathcal{G}_\eta)$ as the probability of sampling an effective configuration.

We model self-correcting proposal refinement as a distribution-level update

$$C_\gamma(q) = (q + \gamma q^*) / (1 + \gamma), \quad (13)$$

where $\gamma \geq 0$ is the correction strength and q^* is a utility-biased reference distribution, instantiated in Appendix D as $q_{\tau,\beta}^*(c) \propto \exp(\beta U_\tau(c))$. For the effective set, this update gives

$$C_\gamma(q)(\mathcal{G}_\eta) - q(\mathcal{G}_\eta) = \gamma(q^*(\mathcal{G}_\eta) - q(\mathcal{G}_\eta))/(1 + \gamma). \quad (14)$$

Thus, self-correction increases the probability of sampling an effective attack whenever the reference distribution is more concentrated on $\mathcal{G}_\eta$ than the current proposal distribution. As a clean special case, if $q^*(\mathcal{G}_\eta) = 1$, the probability mass outside $\mathcal{G}_\eta$ contracts by a factor of $1/(1 + \gamma)$.

This mass improvement translates into finite-budget search efficiency. If a round samples b candidates from q_t , the probability that the batch contains at least one effective configuration is

$$h_t = 1 - (1 - p_t)^b. \quad (15)$$

Since h_t is monotone increasing in p_t , any update that increases $q_t(\mathcal{G}_\eta)$ improves the probability of discovering an effective attack in the next batch. If $p_t \geq p > 0$ across rounds, the expected hitting time satisfies

$$\mathbb{E}[T] \leq 1/(1 - (1 - p)^b). \quad (16)$$

Therefore, the theory does not claim global attack optimization. It shows that RGAR and SCAS improve finite-budget attack search whenever they shift sufficient proposal mass toward high-utility attack configurations, with noisy and finite-sample cases treated in Appendix D.

5 Experiments

5.1 Experimental Setup

Datasets and victim models. We evaluate **WMAttack** on Atari [6] and DeepMind Control Suite (DMC) [42]. The main experiments use DreamerV3 [22] as the primary victim model, covering 26 Atari tasks and 20 DMC tasks. To examine cross-model applicability, we further report additional results on DreamerV2 [21], TD-MPC2 [23], and IRIS [37]. All victim agents are trained before evaluation and remain fixed during adversarial testing.

Attack search space. WMAttack searches over multiple attack families: APGD-CE and APGD-DLR [8], FAB [9], Square Attack [2], and PhysCond-WMA [17]. For each task and attack family, the configuration space includes the perturbation budget ϵ , optimization steps, restarts, ρ , random seed, and allocation rule. Each candidate configuration is evaluated through closed-loop rollouts and scored using the scalarized utility defined in Section 3.3. All compared methods use the same victim checkpoints, search space, rollout protocol, and evaluation budget within each setting.

Baselines and ablations. We compare WMAttack against two baselines. **Random** samples attack configurations without retrieval or feedback-conditioned refinement. **Claudini** is a Claudini-style autoresearch baseline adapted from Claudini [38]; it uses iterative feedback to propose attack configurations but does not use RGAR or cross-task latent retrieval. We also conduct ablations to isolate RGAR and evaluate different LLM backbones for proposal generation, including Qwen [43] and LLaMA [12].

Metrics. We report both attack strength and search efficiency. **Drop** is normalized reward degradation, **Flip** is the action flip rate, and **Utility** is the scalarized attack objective. **Time** reports wall-clock evaluation time, while threshold-based efficiency measures how often a method reaches a target fraction of its final best utility. Detailed task lists, hyperparameter ranges, rollout budgets, and implementation details are provided in Appendix B.

5.2 Main Results

Table 1 summarizes the primary adversarial evaluation results across multiple suites and model architectures. Following Panfilov et al. [38], we compare WMAttack against two baselines: *Random*, which performs search without retrieval or feedback-conditioned refinement, and an adapted *Claudini*-style autoresearch baseline. For DreamerV3 Atari, the main results use four attack families because Square Attack is substantially more expensive in the Atari closed-loop setting and was therefore excluded from the primary Atari runs; DMC includes all five attack families.

Table 1: Main adversarial evaluation results.

Victim	Suite	Tasks	Attacks	Method	Drop $\uparrow$	Flip $\uparrow$	Utility $\uparrow$	Time (min)
DreamerV3	Atari	26	4	Claudini	0.497	0.400	-0.132	77.2
DreamerV3	Atari	26	4	Random	0.446	0.377	-0.163	73.4
DreamerV3	Atari	26	4	WMAttack	1.034	0.660	0.430	106.2
DreamerV3	DMC	20	5	Claudini	0.319	0.705	-0.137	55.4
DreamerV3	DMC	20	5	Random	0.352	0.721	-0.123	59.0
DreamerV3	DMC	20	5	WMAttack	0.682	0.800	0.174	120.2
DreamerV2	Atari	10	4	WMAttack	0.539	0.554	-0.021	242.9
DreamerV2	DMC	10	5	WMAttack	0.397	0.762	0.046	253.5
TD-MPC2	DMC	10	5	WMAttack	0.621	0.786	0.152	68.1
IRIS	Atari	10	4	WMAttack	0.550	0.474	-0.214	830.1

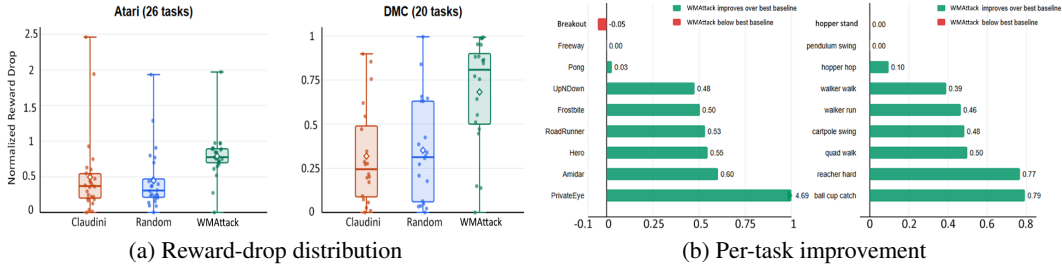


Figure 3: Task-level reward-drop distribution and per-task improvement on DreamerV3 Atari and DMC. WMAttack shifts the reward-drop distribution upward and improves over the stronger baseline on most displayed tasks.

Superior Adversarial Potency on DreamerV3. WMAttack significantly outperforms both baselines on the DreamerV3 agent. On Atari, WMAttack achieves an average normalized reward drop of 1.034, surpassing Random and Claudini by 131.8% and 108.0%, respectively. Notably, WMAttack is the only method to yield a positive scalarized utility 0.430, indicating that the strength of the discovered attacks justifies the search cost. Similar trends are observed on DMC, where WMAttack reaches a reward drop of 0.682 and the highest action flip rate 0.800. These results validate that the synergistic combination of RGAR and SCAS identifies substantially more potent adversarial configurations than either random exploration or feedback-only search.

Cross-model generalization. We further evaluate WMAttack on DreamerV2, TD-MPC2, and IRIS to test whether the attack-search framework extends beyond the primary DreamerV3 setting. WMAttack exposes clear vulnerabilities across these model families, achieving reward drops of 0.539 on DreamerV2 Atari, 0.397 on DreamerV2 DMC, 0.621 on TD-MPC2 DMC, and 0.550 on IRIS Atari. The scalarized utility differs across victims because it also accounts for evaluation cost; for example, IRIS obtains a high reward drop but negative utility due to its much larger rollout runtime. These results suggest that WMAttack can be applied to different world-model architectures, while also revealing architecture-dependent strength–cost trade-offs.

Consistency and attack-family selection. Figure 3 provides a task-level view of the DreamerV3 results. Panel (a) shows that WMAttack shifts the reward-drop distribution upward relative to both baselines, indicating stronger attacks across the task distribution. Panel (b) further shows that WMAttack improves over the stronger baseline on most tasks, achieving the highest reward drop on 24 out of 26 Atari tasks and 18 out of 20 DMC tasks. These results indicate that the gains are consistent across environments rather than driven by a small number of outliers. The strongest attack family also varies across suites: APGD-CE is selected most often on Atari, whereas FAB dominates on DMC. This heterogeneity motivates searching over multiple attack families instead of relying on a single fixed attack. Detailed attack-family selection statistics are provided in Appendix E.

Table 2: Threshold-based search efficiency. Hit Rate reports the fraction of task–attack pairs that reach 90% of their final best utility.

Victim	Suite	Method	Pairs	Hit Rate $\uparrow$	Trials $\downarrow$	Time (s)
DreamerV3	Atari	Claudini	104	0.538	3.36	349
DreamerV3	Atari	Random	104	0.529	3.67	394
DreamerV3	Atari	WMAttack	104	0.875	3.45	491
DreamerV3	DMC	Claudini	100	0.170	3.53	254
DreamerV3	DMC	Random	100	0.180	3.44	325
DreamerV3	DMC	WMAttack	100	0.490	3.71	823
DreamerV2	Atari	WMAttack	40	0.545	2.47	111
DreamerV2	DMC	WMAttack	50	0.213	2.69	232
TD-MPC2	DMC	WMAttack	50	0.550	2.27	339
IRIS	Atari	WMAttack	40	0.000	—	—

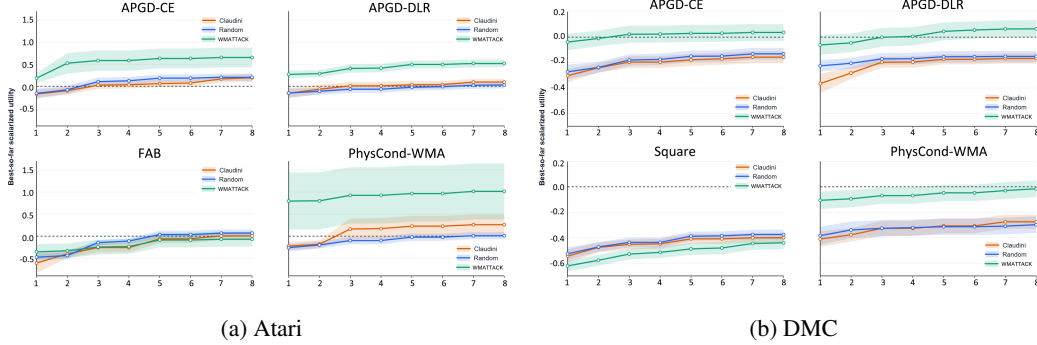


Figure 4: Attack-specific search efficiency on DreamerV3 Atari and DMC. WMAttack generally reaches higher best-so-far utility earlier than Claudini and Random across most attack families.

5.3 Efficiency–Accuracy Trade-off

We evaluate whether WMAttack can find strong attacks efficiently under a limited search budget. Table 2 reports the fraction of task–attack pairs that reach 90% of their final best utility, while detailed attack-specific efficiency curves are provided in Appendix F.

WMAttack enters high-utility regions more reliably. On DreamerV3 Atari, WMAttack reaches the threshold on 87.5% of task–attack pairs, compared with 52.9% for Random and 53.8% for Claudini. On DMC, the hit rate improves from about 18% for the baselines to 49.0%. Although WMAttack can require more wall-clock time, especially on DMC, it substantially increases the probability of finding useful attacks under the same finite search protocol.

The efficiency gains transfer beyond DreamerV3. WMAttack also reaches nontrivial hit rates on DreamerV2 Atari (54.5%), DreamerV2 DMC (21.3%), and TD-MPC2 DMC (55.0%). For IRIS, the hit rate is zero under this threshold because its final utilities remain negative after accounting for the high rollout cost, even though Table 1 shows nontrivial reward degradation.

5.4 Ablation and Design Analysis

RGAR and SCAS are complementary. Table 3 shows that adding RGAR to SCAS improves first-round Utility / Drop / Flip from 0.048/0.445/0.394 to 0.397/0.770/0.573, confirming that representation-guided retrieval provides a stronger warm start. After search, the full *RGAR + SCAS* variant achieves the best final Utility / Drop / Flip 0.020/0.665/0.568, while *RGAR only* and *w/o RGAR and SCAS* still have negative final utility. This suggests that retrieval improves initialization, but feedback-driven refinement is needed to convert a good warm start into higher final attack utility. Additional attack-family and pair-level analyses are provided in Appendix G.

Larger LLM backbones are not uniformly better. Table 4 shows that attack-search performance does not scale monotonically with model size. Qwen2.5-3B obtains the best reward drop (1.034) and utility (0.430), Qwen2.5-7B obtains the highest flip rate (0.675) but incurs the largest runtime, and

Table 3: Ablation of RGAR and SCAS on DreamerV3 Atari. Each cell reports Utility / Drop / Flip.

Variant	First Utility / Drop / Flip $\uparrow$	Final Utility / Drop / Flip $\uparrow$
SCAS w/o RGAR	0.048 / 0.445 / 0.394	-0.325 / 0.349 / 0.396
RGAR + SCAS	0.397 / 0.770 / 0.573	0.020 / 0.665 / 0.568
RGAR only, w/o SCAS	0.245 / 0.744 / 0.580	-0.046 / 0.645 / 0.572
w/o RGAR and SCAS	0.214 / 0.687 / 0.604	-0.048 / 0.615 / 0.602

Table 4: Sensitivity to the LLM backbone on DreamerV3 Atari.

Model	Size	Drop $\uparrow$	Flip $\uparrow$	Utility $\uparrow$	Trials $\downarrow$	Time (s)
LLaMA	1B	0.715	0.637	0.114	12.18	5528
	3B	0.812	0.668	0.210	12.67	6803
	8B	0.769	0.629	0.171	12.46	5215
Qwen	1.5B	0.754	0.640	0.168	9.76	5164
	3B	1.034	0.660	0.430	10.66	6338
	7B	0.916	0.675	0.314	10.45	8252

Qwen2.5-1.5B is the most efficient but weaker in final attack strength. These results suggest that the LLM backbone acts as a proposal generator, and its effectiveness depends on the strength–cost trade-off rather than parameter count alone.

6 Conclusion

We introduced **WMAttack**, an automated attack-search framework for adversarial evaluation of world-model agents. By combining representation-guided retrieval with self-correcting proposal refinement, WMAttack searches high-impact attack configurations under finite evaluation budgets. Experiments on Atari and DMC show stronger attacks than random search and a Claudini-style baseline, while ablations confirm the benefits of RGAR and feedback-conditioned search.

References

- [1] Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. In *Advances in Neural Information Processing Systems*, 2024.
- [2] Maksym Andriushchenko, Francesco Croce, Nicolas Flammarion, and Matthias Hein. Square attack: A query-efficient black-box adversarial attack via random search. In *European Conference on Computer Vision*, 2020.
- [3] Yarden As, Bhavya Sukhija, and Andreas Krause. Safe exploration using bayesian world models and log-barrier optimization. *arXiv preprint arXiv:2405.05890*, 2024.
- [4] András Balogh and Márk Jelasity. Verification of the implicit world model in a generative model via adversarial sequences. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=BL0IB8CwBI>.
- [5] Vahid Behzadan and Arslan Munir. On the robustness of deep reinforcement learning to adversarial attacks. In *Machine Learning and Data Mining in Pattern Recognition: 13th International Conference, MLDM 2017*, pages 262–274. Springer, 2017.
- [6] Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.
- [7] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choro-manski, Tianli Ding, Danny Driess, Avinash Fu, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of the 7th Annual Conference on Robot Learning (CoRL)*, 2023.

- [8] Francesco Croce and Matthias Hein. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. In *International Conference on Machine Learning*, 2020.
- [9] Francesco Croce and Matthias Hein. Minimally distorted adversarial examples with a fast adaptive boundary attack. In *International Conference on Machine Learning*, 2020.
- [10] Francesco Croce, Maksym Andriushchenko, Vikash Sehwal, Edoardo DeBenedetti, Nicolas Flammarion, Mung Chiang, Prateek Mittal, and Matthias Hein. RobustBench: A standardized adversarial robustness benchmark. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
- [11] Siyuan Duan, Ke Zhang, and Xizhao Luo. Trap: Tail-aware ranking attack for world-model planning. *arXiv preprint arXiv:2605.01950*, 2026.
- [12] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL <https://arxiv.org/abs/2407.21783>.
- [13] Jianjie Fang, Yingshan Lei, Qin Wan, Ziyu Wang, Yuchao Huang, Yongyan Xu, Baining Zhao, Weichen Zhang, Chen Gao, Xinlei Chen, and Yong Li. iworld-bench: A benchmark for interactive world models with a unified action generation framework. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. Originally appeared as arXiv preprint arXiv:2510.xxxxx in 2025.
- [14] Adam Gleave, Michael Dennis, Cody Wild, Neel Kant, Sergey Levine, and Stuart Russell. Adversarial policies: Attacking deep reinforcement learning. In *International Conference on Learning Representations*, 2020.
- [15] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations*, 2015.
- [16] Zhixiang Guo, Siyuan Liang, Aishan Liu, and Dacheng Tao. Copyrightshield: Enhancing diffusion model security against copyright infringement attacks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 19417–19426, 2025.
- [17] Zhixiang Guo, Siyuan Liang, Andras Balogh, Noah Lunberry, Rong-Cheng Tu, Mark Jelasity, and Dacheng Tao. When world models dream wrong: Physical-conditioned adversarial attacks against world models, 2026. URL <https://arxiv.org/abs/2602.18739>.
- [18] David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems*, 2018.
- [19] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning*, 2019.
- [20] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.
- [21] Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. In *International Conference on Learning Representations*, 2021.
- [22] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- [23] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024.
- [24] Anthony Hu, Lloyd Russell, Hudson Yeo, Zak Murez, George Fedoseev, Alex Kendall, Jamie Shotton, and Gianluca Corrado. GAIA-1: A generative world model for autonomous driving. *arXiv preprint arXiv:2309.17080*, 2023.

- [25] Sandy Huang, Nicolas Papernot, Ian Goodfellow, Yan Duan, and Pieter Abbeel. Adversarial attacks on neural network policies. In *International Conference on Learning Representations Workshop*, 2017.
- [26] Weidong Huang, Jiaming Ji, Chunhe Xia, Borong Zhang, and Yaodong Yang. Safedreamer: Safe reinforcement learning with world models. In *International Conference on Learning Representations*, 2024.
- [27] Taisuke Kobayashi. Lira: Light-robust adversary for model-based reinforcement learning in real world. *Robotics and Autonomous Systems*, 2025.
- [28] Dehong Kong, Sifan Yu, Siyuan Liang, Jiawei Liang, Jianhou Gan, Aishan Liu, and Wenqi Ren. Universal camouflage attack on vision-language models for autonomous driving. *arXiv preprint arXiv:2509.20196*, 2025.
- [29] Muxue Liang, Chuan Wang, Siyuan Liang, Aishan Liu, Yanan Cao, Qingyong Li, Zeming Liu, Liang Yang, and Xiaochun Cao. Hard-label black-box adversarial attacks for implicit scene interactions. *IEEE Transactions on Information Forensics and Security*, 20:10346–10360, 2025.
- [30] Siyuan Liang, Longkang Li, Yanbo Fan, Xiaojun Jia, Jingzhi Li, Baoyuan Wu, and Xiaochun Cao. A large-scale multiple-objective method for black-box attack against object detection. In *European Conference on Computer Vision*, 2022.
- [31] Siyuan Liang, Baoyuan Wu, Yanbo Fan, Xingxing Wei, and Xiaochun Cao. Parallel rectangle flip attack: A query-based black-box attack against object detection. *arXiv preprint arXiv:2201.08970*, 2022.
- [32] Siyuan Liang, Wei Wang, Ruoyu Chen, Aishan Liu, Boxi Wu, Ee-Chien Chang, Xiaochun Cao, and Dacheng Tao. Object detectors in the open environment: Challenges, solutions, and outlook. *arXiv preprint arXiv:2403.16271*, 2024.
- [33] Yen-Chen Lin, Zhang-Wei Hong, Yuan-Hong Liao, Meng-Li Shih, Ming-Yu Liu, and Min Sun. Tactics of adversarial attack on deep reinforcement learning agents. In *International Joint Conference on Artificial Intelligence*, 2017.
- [34] Aishan Liu, Jiakai Wang, Tianyuan Zhang, Hainan Li, Jiangfan Liu, Siyuan Liang, Yilong Ren, Xianglong Liu, and Dacheng Tao. Metadv: A unified and interactive adversarial testing platform for autonomous driving. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 13474–13476, 2025.
- [35] Jiayang Liu, Siyu Zhu, Siyuan Liang, Jie Zhang, Han Fang, Weiming Zhang, and Ee-Chien Chang. Improving adversarial transferability by stable diffusion. *arXiv preprint arXiv:2311.11017*, 2023.
- [36] Kuanrong Liu, Siyuan Liang, Cheng Qian, Ming Zhang, and Xiaochun Cao. Bridging the task gap: Multi-task adversarial transferability in clip and its derivatives. In *Chinese Conference on Pattern Recognition and Computer Vision (PRCV)*, pages 152–166. Springer, 2025.
- [37] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations*, 2023.
- [38] Alexander Panfilov, Peter Romov, Igor Shilov, Yves-Alexandre de Montjoye, Jonas Geiping, and Maksym Andriushchenko. Claudini: Autoresearch discovers state-of-the-art adversarial attack algorithms for llms. *arXiv preprint arXiv:2603.24511*, 2026.
- [39] Waris Radji and Odalric-Ambrym Maillard. How hard is it to confuse a world model? *arXiv preprint arXiv:2510.21232*, 2025.
- [40] Junwon Seo, Kensuke Nakamura, and Andrea Bajcsy. Uncertainty-aware latent safety filters for avoiding out-of-distribution failures. *arXiv preprint arXiv:2505.00779*, 2025.
- [41] Ruixiang Sun, Hongyu Zang, Xin Li, and Riashat Islam. Learning latent dynamic robust representations for world models. *arXiv preprint arXiv:2405.06263*, 2024.

- [42] Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Riedmiller. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.
- [43] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. URL <https://arxiv.org/abs/2412.15115>.
- [44] Rishi Upadhyay, Howard Zhang, Jim Solomon, Ayush Agrawal, Pranay Boreddy, Shruti Satya Narayana, Yunhao Ba, Alex Wong, Celso M. de Melo, and Achuta Kadambi. World-Bench: Disambiguating physics for diagnostic evaluation of world models. *arXiv preprint arXiv:2601.21282*, 2026.
- [45] Keyon Vafa, Justin Y. Chen, Ashesh Rambachan, Jon Kleinberg, and Sendhil Mullainathan. Evaluating the world model implicit in a generative model. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [46] Lu Wang, Tianyuan Zhang, Yang Qu, Siyuan Liang, Yuwei Chen, Aishan Liu, Xianglong Liu, and Dacheng Tao. Black-box adversarial attack on vision language models for autonomous driving. *arXiv preprint arXiv:2501.13563*, 2025.
- [47] Wenqiang Wang, Siyuan Liang, Xiao Yan, and Xiaochun Cao. Text adversarial attacks with dynamic outputs. *arXiv preprint arXiv:2509.22393*, 2025.
- [48] Zhiyuan Wang, Zeliang Zhang, Siyuan Liang, and Xiaosen Wang. Diversifying the high-level features for better adversarial transferability. *arXiv preprint arXiv:2304.10136*, 2023.
- [49] Xingxing Wei, Siyuan Liang, Ning Chen, and Xiaochun Cao. Transferable adversarial attacks for image and video object detection. *arXiv preprint arXiv:1811.12641*, 2018.
- [50] Shuhan Xu, Siyuan Liang, Hongling Zheng, Yong Luo, Han Hu, Lefei Zhang, and Dacheng Tao. Ctrlattack: A unified attack on world-model control in diffusion models. *arXiv preprint arXiv:2603.13435*, 2026.
- [51] Sherry Yang, Yilun Du, Kamyar Ghasemipour, Jonathan Tompson, Leslie Kaelbling, Dale Schuurmans, and Pieter Abbeel. Learning interactive real-world simulators. In *International Conference on Learning Representations*, 2024.
- [52] Chenlu Ye, Jiafan He, Quanquan Gu, and Tong Zhang. Towards robust model-based reinforcement learning against adversarial corruption. *arXiv preprint arXiv:2402.08991*, 2024.
- [53] Amy Zhang, Rowan McAllister, Roberto Calandra, Yarin Gal, and Sergey Levine. Learning invariant representations for reinforcement learning without reconstruction. In *International Conference on Learning Representations (ICLR)*, 2021.
- [54] Harry Zhang. Safe deep model-based reinforcement learning with lyapunov functions. *arXiv preprint arXiv:2405.16184*, 2024.
- [55] Huan Zhang, Hongge Chen, Chaowei Xiao, Bo Li, Duane Boning, and Cho-Jui Hsieh. Robust deep reinforcement learning against adversarial perturbations on state observations. In *Advances in Neural Information Processing Systems*, 2020.
- [56] Tianyuan Zhang, Lu Wang, Xinwei Zhang, Yitong Zhang, Boyi Jia, Siyuan Liang, Shengshan Hu, Qiang Fu, Aishan Liu, and Xianglong Liu. Visual adversarial attack on vision-language models for autonomous driving. *arXiv preprint arXiv:2411.18275*, 2024.
- [57] Weipu Zhang, Gang Wang, Jian Sun, Yetian Yuan, and Gao Huang. STORM: Efficient stochastic transformer based world models for reinforcement learning. *arXiv preprint arXiv:2310.09615*, 2024.
- [58] Geigh Zollicoffer, Tanush Chopra, Mingkuan Yan, Xiaoxu Ma, Kenneth Eaton, and Mark Riedl. World model robustness via surprise recognition. *arXiv preprint arXiv:2512.01119*, 2025.

A Victim and Threat Model Details

This appendix details the closed-loop evaluation process under clean and attacked observations. The victim is a fixed world-model agent

$$M_\theta = (\phi_\theta, f_\theta, \pi_\theta), \quad (17)$$

where ϕ_θ is the encoder or representation module, f_θ is the latent dynamics model, and π_θ is the policy.

Under clean evaluation, the agent interacts with the environment as

$$z_t = \phi_\theta(o_{\leq t}, u_{< t}), \quad u_t \sim \pi_\theta(\cdot \mid z_t), \quad o_{t+1}, r_t \sim \mathcal{E}(\cdot \mid o_t, u_t), \quad (18)$$

where $\mathcal{E}$ denotes the environment dynamics and reward process. Under adversarial evaluation, the environment dynamics remain unchanged, but the agent receives perturbed observations:

$$\tilde{z}_t = \phi_\theta(\tilde{o}_{\leq t}, \tilde{u}_{< t}), \quad \tilde{u}_t \sim \pi_\theta(\cdot \mid \tilde{z}_t), \quad o_{t+1}, r_t^c \sim \mathcal{E}(\cdot \mid o_t, \tilde{u}_t). \quad (19)$$

Thus, attacks affect returns only indirectly by changing the observations used to construct latent states and actions.

In WMAttack, attacks are applied to image observation keys used by the world-model encoder. Let $\mathcal{K}_{\text{img}}$ denote this set. For each $k \in \mathcal{K}_{\text{img}}$, the attacked observation is

$$\tilde{o}_{t,k} = \Pi_{[-0.5, 0.5]}(o_{t,k} + \delta_{t,k}), \quad \|\delta_{t,k}\|_\infty \leq \epsilon/255. \quad (20)$$

The projection keeps inputs within the normalized observation range, and ϵ is specified in pixel scale. Non-image keys are left unchanged:

$$\tilde{o}_{t,k} = o_{t,k}, \quad k \notin \mathcal{K}_{\text{img}}. \quad (21)$$

The attacker may choose the attack configuration $c \in \mathcal{C}_\tau$, including the attack family and hyperparameters, but cannot modify the environment dynamics, reward function, policy parameters, world-model parameters, checkpoint, or training data.

B Attack Configuration Space

This appendix provides additional details on the attack configuration space used by WMAttack. For each task and attack family, WMAttack searches over a discrete set of attack configurations. Each configuration contains the attack type, perturbation budget ϵ , number of optimization steps, number of restarts, ρ , random seed, and allocation rule. The perturbation budget ϵ is specified in pixel scale and converted to $\epsilon/255$ in the normalized observation space.

WMAttack evaluates five attack families:

- **APGD-CE**: an adaptive projected-gradient attack using cross-entropy loss.
- **APGD-DLR**: an adaptive projected-gradient attack using a difference-of-logits-ratio style objective.
- **FAB**: a boundary-oriented attack adapted to the world-model policy output.
- **Square Attack**: a black-box patch-based attack that searches over square perturbation regions.
- **PhysCond-WMA**: a physics-conditioned world-model attack designed to exploit world-model-specific latent and policy sensitivities.

The search spaces used in our implementation are summarized in Table 5. Exact values may be adjusted for different hardware or evaluation budgets, but all compared methods use the same search space.

Table 5: Attack search spaces used by WMAttack.

Attack	ϵ candidates	Step candidates	Allocation modes
APGD-CE	2–20	4–24	fixed, margin-linear
APGD-DLR	2–20	4–24	fixed, margin-linear
FAB	2–20	6–32	fixed, margin-linear
PhysCond-WMA	2–20	6–32	fixed, margin-linear
Square	2–16	20–160	fixed, margin-linear

For the fixed allocation mode, WMAttack applies the same attack budget at each evaluated decision point. For margin-linear allocation, WMAttack adaptively adjusts the effective number of attack steps and perturbation budget according to the policy margin. This allows the evaluator to allocate more attack effort to states where the policy is more confident and less effort to states where the action is already unstable.

The remaining configuration fields, including restarts, ρ , and random seeds, follow the evaluator configuration used for each run. They are kept consistent across WMAttack and all baselines to ensure that performance differences come from the search strategy rather than from different attack spaces.

C Utility Components

This appendix provides the detailed definitions of the utility components used in WMAttack. For a task τ , let $J_{\text{clean}}(\tau)$ denote the expected episodic return under clean observations, and let $J_{\text{adv}}(\tau, c)$ denote the expected attacked return under attack configuration c .

The normalized reward degradation is defined as

$$D_{\tau}(c) = \frac{J_{\text{clean}}(\tau) - J_{\text{adv}}(\tau, c)}{|J_{\text{clean}}(\tau)| + 1}. \quad (22)$$

This term is larger when the attack causes a stronger drop in episodic return. The denominator normalizes the score across tasks with different reward scales and avoids division by zero when the clean return is close to zero.

The action-instability term $F_{\tau}(c)$ measures how often the adversarial observation changes the agent’s selected action. For discrete action spaces, it is computed as the fraction of evaluated decision points where

$$\arg \max_u \pi_{\theta}(u \mid z_t) \neq \arg \max_u \pi_{\theta}(u \mid \tilde{z}_t). \quad (23)$$

For continuous action spaces, WMAttack uses a thresholded action shift:

$$\mathbb{I} [\|u_t - \tilde{u}_t\|_1 > \kappa], \quad (24)$$

where κ is a small threshold. These per-step measurements are averaged over rollout steps and episodes to obtain $F_{\tau}(c)$.

The runtime term $T_{\tau}(c)$ denotes the wall-clock time required to evaluate configuration c . It is penalized through $\log(1 + T_{\tau}(c))$ so that expensive attacks are discouraged while avoiding an overly large penalty for moderate runtime differences.

The rollout-variability term is defined as

$$V_{\tau}(c) = \frac{\text{Std}[R_{\text{adv}}(c)]}{|J_{\text{clean}}(\tau)| + 1}, \quad (25)$$

where $R_{\text{adv}}(c)$ denotes episodic returns from repeated attacked rollouts under configuration c . This term penalizes configurations whose outcomes are highly variable across repeated evaluations.

Combining these components, WMAttack uses the scalarized utility

$$U_{\tau}(c) = D_{\tau}(c) + w_f F_{\tau}(c) - w_r \log(1 + T_{\tau}(c)) - w_v V_{\tau}(c). \quad (26)$$

In our implementation, we use $w_f = 0.25$, $w_r = 0.15$, and $w_v = 0.05$. Higher utility corresponds to stronger reward degradation, higher action instability, lower evaluation time, and lower rollout variability.

D Theoretical Details

This appendix provides the derivations omitted from Section 4.4. We use the same notation as in the main text: $\mathcal{C}_\tau$ is the finite attack configuration space, $U_\tau(c)$ is the scalarized utility, and $\mathcal{G}_\eta$ is the set of η -effective configurations.

D.1 Utility-Biased Reference Distribution

Section 4.4 uses a reference distribution q^* that assigns larger probability to high-utility attacks. A concrete construction is the Gibbs distribution

$$q_{\tau,\beta}^*(c) = \frac{\exp(\beta U_\tau(c))}{\sum_{c' \in \mathcal{C}_\tau} \exp(\beta U_\tau(c'))}, \quad c \in \mathcal{C}_\tau, \quad (27)$$

where $\beta \geq 0$ is an inverse-temperature parameter. When $\beta = 0$, this distribution is uniform; as β increases, it concentrates more mass on high-utility configurations. Let

$$r = q_{\tau,\beta}^*(\mathcal{G}_\eta) \quad (28)$$

denote the reference mass on the effective set.

For the self-correction operator

$$C_\gamma(q) = (q + \gamma q^*) / (1 + \gamma), \quad (29)$$

the effective-set mass is

$$C_\gamma(q)(\mathcal{G}_\eta) = \frac{q(\mathcal{G}_\eta) + \gamma r}{1 + \gamma}. \quad (30)$$

Therefore,

$$C_\gamma(q)(\mathcal{G}_\eta) - q(\mathcal{G}_\eta) = \frac{\gamma}{1 + \gamma} (r - q(\mathcal{G}_\eta)). \quad (31)$$

Thus, correction improves the probability of sampling an effective attack whenever $r > q(\mathcal{G}_\eta)$. The oracle case in the main text corresponds to $r = 1$, which gives

$$1 - C_\gamma(q)(\mathcal{G}_\eta) = \frac{1 - q(\mathcal{G}_\eta)}{1 + \gamma}. \quad (32)$$

D.2 Noisy Correction

The realized update need not exactly equal $C_\gamma(q_t)$. Suppose the next proposal distribution satisfies the sufficient noisy-correction condition

$$q_{t+1}(\mathcal{G}_\eta) \geq C_\gamma(q_t)(\mathcal{G}_\eta) - \xi_t, \quad (33)$$

where $\xi_t \geq 0$ captures approximation error from retrieval, feedback, LLM generation, and finite rollout estimates. Let $p_t = q_t(\mathcal{G}_\eta)$ and $r = q^*(\mathcal{G}_\eta)$. Then

$$q_{t+1}(\mathcal{G}_\eta) \geq \frac{p_t + \gamma r}{1 + \gamma} - \xi_t. \quad (34)$$

Hence $q_{t+1}(\mathcal{G}_\eta) > p_t$ whenever

$$\xi_t < \frac{\gamma}{1 + \gamma} (r - p_t). \quad (35)$$

This condition is sufficient, not automatic: if the correction signal is too noisy or not more concentrated on $\mathcal{G}_\eta$ than the current proposal, the theory does not claim improvement.

D.3 Comparison with a Generic LLM Baseline

Consider WMAttack and a generic LLM-based search baseline under the same evaluator, task, search space, and budget. Let their effective correction strengths be γ_{ours} and γ_{base} , respectively. Assume both methods start from the same proposal mass $p_t = q_t(\mathcal{G}_\eta)$ and are compared against the same reference mass $r = q^*(\mathcal{G}_\eta)$.

The difference between their ideal corrected masses is

$$\begin{aligned} C_{\gamma_{\text{ours}}}(q_t)(\mathcal{G}_\eta) - C_{\gamma_{\text{base}}}(q_t)(\mathcal{G}_\eta) &= \frac{p_t + \gamma_{\text{ours}}r}{1 + \gamma_{\text{ours}}} - \frac{p_t + \gamma_{\text{base}}r}{1 + \gamma_{\text{base}}} \\ &= \frac{(\gamma_{\text{ours}} - \gamma_{\text{base}})(r - p_t)}{(1 + \gamma_{\text{ours}})(1 + \gamma_{\text{base}})}. \end{aligned} \quad (36)$$

If the realized proposal updates have approximation errors ξ_{ours} and ξ_{base} , then WMAttack assigns more mass to $\mathcal{G}_\eta$ than the baseline whenever

$$\xi_{\text{ours}} + \xi_{\text{base}} < \frac{(\gamma_{\text{ours}} - \gamma_{\text{base}})(r - p_t)}{(1 + \gamma_{\text{ours}})(1 + \gamma_{\text{base}})}, \quad (37)$$

provided $\gamma_{\text{ours}} > \gamma_{\text{base}}$ and $r > p_t$. This formalizes a sufficient condition under which representation-guided retrieval and feedback improve the one-round probability of hitting an effective attack compared with a generic LLM proposal mechanism.

D.4 Finite-Episode Utility Estimation

The main analysis uses the population utility $U_\tau(c)$, while experiments observe an empirical estimate $\hat{U}_\tau(c)$ from a finite number of rollout episodes. Assume episodic returns lie in $[R_{\min}, R_{\max}]$ and action-flip indicators are Bernoulli. Ignoring deterministic runtime terms and absorbing variability-estimation error into the same bound, Hoeffding’s inequality and a union bound over $\mathcal{C}_\tau$ imply that, with probability at least $1 - \delta$,

$$\left| \hat{U}_\tau(c) - U_\tau(c) \right| \leq \zeta_m(\delta) \quad \text{for all } c \in \mathcal{C}_\tau, \quad (38)$$

where

$$\zeta_m(\delta) = \frac{R_{\max} - R_{\min}}{|J_{\text{clean}}(\tau)| + 1} \sqrt{\frac{\log(4|\mathcal{C}_\tau|/\delta)}{2m}} + w_f \sqrt{\frac{\log(4|\mathcal{C}_\tau|/\delta)}{2m}}. \quad (39)$$

Consequently, if a configuration is empirically η -optimal,

$$\hat{U}_\tau(c) \geq \max_{c' \in \mathcal{C}_\tau} \hat{U}_\tau(c') - \eta, \quad (40)$$

then it is population $(\eta + 2\zeta_m(\delta))$ -optimal:

$$U_\tau(c) \geq U_\tau^* - \eta - 2\zeta_m(\delta). \quad (41)$$

Thus, finite-episode evaluation enlarges the effective-set tolerance by a statistical estimation term rather than invalidating the search analysis.

E Task-Level Main Results

This appendix expands the main results with task-level statistics. Unless otherwise specified, the Atari WMAAttack rows use the Qwen2.5-3B backend selected in the main text, while the DMC rows use the main DreamerV3 DMC run. Rows marked with “—” indicate that the corresponding attack family was not available in the current export for that suite.

Table 6: Attack-family selection counts for the strongest WMAAttack configuration on each task. For Atari, Square Attack is listed for completeness but is excluded from the primary Atari runs due to its closed-loop evaluation cost.

Suite	APGD-CE	APGD-DLR	FAB	Square	PhysCond-WMA
Atari	17	3	2	0	4
DMC	3	1	14	0	2

Table 7: Task-level comparison on DreamerV3 Atari.

Task	Claudini D	Random D	WMAAttack D	Claudini U	Random U	WMAAttack U
Alien	0.380	0.447	0.671	-0.142	-0.115	0.195
Amidar	0.296	0.229	0.899	-0.382	-0.408	0.366
Assault	0.036	0.090	0.519	-0.587	-0.560	-0.098
Asterix	0.603	0.397	0.775	0.088	-0.105	0.370
BankHeist	0.748	0.797	0.974	0.074	0.121	0.459
BattleZone	0.473	0.371	0.710	-0.146	-0.257	0.106
Boxing	0.630	0.698	0.973	0.021	0.056	0.332
Breakout	0.927	0.906	0.879	0.565	0.538	0.370
ChopperCommand	0.421	0.207	0.842	-0.170	-0.392	0.330
CrazyClimber	0.126	0.302	0.755	-0.690	-0.443	0.032
DemonAttack	0.362	0.256	0.740	-0.188	-0.338	0.002
Freeway	0.000	0.000	0.000	-0.628	-0.614	-0.621
Frostbite	0.246	0.202	0.751	-0.285	-0.305	0.068
Gopher	0.531	0.235	0.846	-0.199	-0.495	0.245
Hero	0.014	0.096	0.643	-0.554	-0.484	0.229
Jamesbond	0.185	0.370	0.840	-0.390	-0.189	0.076
Kangaroo	0.549	0.477	0.892	-0.166	-0.199	0.317
Krull	0.169	0.156	0.611	-0.560	-0.554	-0.090
KungFuMaster	0.440	0.311	0.903	-0.269	-0.355	0.072
MsPacman	0.388	0.439	0.697	-0.211	-0.055	0.193
Pong	1.942	1.935	1.971	1.401	1.403	1.453
PrivateEye	2.460	1.287	7.146	1.571	0.459	5.984
Qbert	0.197	0.771	0.966	-0.526	0.188	0.505
RoadRunner	0.354	0.255	0.887	-0.264	-0.399	0.396
Seaquest	0.220	0.137	0.275	-0.404	-0.418	-0.295
UpNDown	0.217	0.233	0.709	-0.394	-0.326	0.196

Table 8: Task-level comparison on DreamerV3 DMC.

Task	Claudini D	Random D	WMAAttack D	Claudini U	Random U	WMAAttack U
acrobot swingup	0.472	0.657	0.986	-0.175	-0.003	0.235
ball in cup catch	0.074	0.049	0.865	-0.366	-0.368	0.264
cartpole balance	0.273	0.319	0.510	-0.158	-0.153	0.071
cartpole balance sparse	0.277	0.647	0.953	-0.265	-0.074	0.356
cartpole swingup	0.202	0.273	0.755	-0.282	-0.266	0.180
cartpole swingup sparse	0.756	0.632	0.988	0.220	0.018	0.455
cheetah run	0.620	0.631	0.845	0.225	0.225	0.445
finger spin	0.545	0.425	0.882	-0.128	-0.232	0.096
finger turn easy	0.197	0.177	0.553	-0.166	-0.209	0.110
finger turn hard	0.171	0.209	0.472	-0.359	-0.265	0.021
hopper hop	0.854	0.839	0.950	0.457	0.437	0.590
hopper stand	0.898	0.996	0.993	0.334	0.514	0.468
pendulum swingup	0.000	0.000	0.000	-0.517	-0.547	-0.629
quadruped run	0.346	0.339	0.643	0.012	-0.009	0.307
quadruped walk	0.217	0.386	0.884	-0.141	-0.016	0.184
reacher easy	0.009	0.037	0.139	-0.357	-0.351	-0.285
reacher hard	0.093	0.064	0.860	-0.286	-0.342	0.424
walker run	0.287	0.307	0.772	-0.116	-0.097	0.384
walker stand	0.027	0.022	0.149	-0.338	-0.383	-0.272
walker walk	0.057	0.034	0.447	-0.332	-0.343	0.069

Table 9: Best discovered WMAAttack configuration for each DreamerV3 Atari task.

Task	Attack	ϵ	Steps	Alloc.	Drop $\uparrow$	Flip $\uparrow$	Utility $\uparrow$
Alien	APGD-CE	20	24	margin-linear	0.671	0.684	0.195
Amidar	APGD-CE	20	24	fixed	0.899	0.686	0.366
Assault	APGD-CE	8	10	margin-linear	0.519	0.521	-0.098
Asterix	APGD-DLR	8	6	margin-linear	0.775	0.623	0.370
BankHeist	APGD-CE	20	20	margin-linear	0.974	0.743	0.459
BattleZone	APGD-CE	8	10	margin-linear	0.710	0.704	0.106
Boxing	APGD-CE	20	20	fixed	0.973	0.782	0.332
Breakout	APGD-DLR	10	12	margin-linear	0.879	0.476	0.370
ChopperCommand	APGD-CE	20	4	fixed	0.842	0.570	0.330
CrazyClimber	APGD-CE	8	6	margin-linear	0.755	0.706	0.032
DemonAttack	PhysCond-WMA	22	18	margin-linear	0.740	0.815	0.002
Freeway	FAB	10	18	margin-linear	0.000	0.524	-0.621
Frostbite	PhysCond-WMA	22	24	margin-linear	0.751	0.838	0.068
Gopher	APGD-CE	8	6	margin-linear	0.846	0.592	0.245
Hero	APGD-CE	20	24	margin-linear	0.643	0.761	0.229
Jamesbond	PhysCond-WMA	22	18	margin-linear	0.840	0.781	0.076
Kangaroo	APGD-CE	22	22	margin-linear	0.892	0.570	0.317
Krull	APGD-CE	20	24	margin-linear	0.611	0.664	-0.090
KungFuMaster	PhysCond-WMA	26	14	margin-linear	0.903	0.837	0.072
MsPacman	APGD-CE	20	10	margin-linear	0.697	0.634	0.193
Pong	FAB	10	12	margin-linear	1.971	0.322	1.453
PrivateEye	APGD-CE	10	12	margin-linear	7.146	0.702	5.984
Qbert	APGD-DLR	8	6	margin-linear	0.966	0.567	0.505
RoadRunner	APGD-CE	22	22	margin-linear	0.887	0.703	0.396
Seaquest	APGD-CE	10	16	margin-linear	0.275	0.646	-0.295
UpNDown	APGD-CE	8	6	margin-linear	0.709	0.720	0.196

Table 10: Best discovered WMAAttack configuration for each DreamerV3 DMC task.

Task	Attack	ϵ	Steps	Alloc.	Drop $\uparrow$	Flip $\uparrow$	Utility $\uparrow$
acrobot swingup	APGD-DLR	10	14	margin-linear	0.986	0.328	0.235
ball in cup catch	FAB	10	18	fixed	0.865	0.657	0.264
cartpole balance	FAB	10	20	fixed	0.510	0.825	0.071
cartpole balance sparse	APGD-CE	10	10	margin-linear	0.953	0.548	0.356
cartpole swingup	FAB	10	20	fixed	0.755	0.718	0.180
cartpole swingup sparse	APGD-CE	10	12	margin-linear	0.988	0.704	0.455
cheetah run	FAB	10	16	fixed	0.845	0.960	0.445
finger spin	PhysCond-WMA	10	10	margin-linear	0.882	0.511	0.096
finger turn easy	FAB	10	18	fixed	0.553	0.937	0.110
finger turn hard	FAB	10	20	fixed	0.472	0.932	0.021
hopper hop	FAB	10	18	fixed	0.950	0.950	0.590
hopper stand	FAB	10	18	margin-linear	0.993	0.830	0.468
pendulum swingup	APGD-CE	10	10	margin-linear	0.000	0.457	-0.629
quadruped run	FAB	10	14	fixed	0.643	0.994	0.307
quadruped walk	PhysCond-WMA	10	18	margin-linear	0.884	1.000	0.184
reacher easy	FAB	10	20	fixed	0.139	0.943	-0.285
reacher hard	FAB	10	16	fixed	0.860	0.904	0.424
walker run	FAB	10	16	fixed	0.772	0.942	0.384
walker stand	FAB	10	20	fixed	0.149	0.919	-0.272
walker walk	FAB	10	14	fixed	0.447	0.949	0.069

Table 11: Preliminary DreamerV2 Atari task-level results.

Task	Attack	ϵ	Steps	Alloc.	Drop $\uparrow$	Flip $\uparrow$	Utility $\uparrow$
Alien	APGD-CE	10	14	margin-linear	0.312	0.650	-0.185
Amidar	APGD-CE	10	14	margin-linear	0.241	0.766	-0.222
Assault	APGD-CE	10	10	margin-linear	0.253	0.386	-0.386
Asterix	APGD-CE	10	14	margin-linear	0.452	0.675	0.118
BankHeist	APGD-CE	10	8	margin-linear	0.950	0.580	0.365

Table 12: Per-task, per-attack-family results on DreamerV3 Atari. Each cell reports Drop / Utility for the best confirmed configuration within an attack family.

Task	APGD-CE	APGD-DLR	FAB	PhysCond-WMA
Alien	0.671 / 0.195	0.611 / 0.048	0.170 / -0.344	0.722 / -0.101
Amidar	0.899 / 0.366	0.722 / 0.094	0.231 / -0.366	0.962 / 0.177
Assault	0.519 / -0.098	0.525 / -0.150	0.213 / -0.388	0.501 / -0.267
Asterix	0.744 / 0.354	0.775 / 0.370	0.214 / -0.257	0.775 / 0.164
BankHeist	0.974 / 0.459	0.973 / 0.423	0.088 / -0.600	0.923 / 0.250
BattleZone	0.710 / 0.106	0.579 / -0.075	0.526 / -0.097	0.803 / -0.052
Boxing	0.973 / 0.332	0.929 / 0.249	0.111 / -0.384	1.002 / 0.044
Breakout	0.739 / 0.083	0.879 / 0.370	0.364 / -0.167	0.933 / 0.300
ChopperCommand	0.842 / 0.330	0.513 / -0.150	0.302 / -0.260	0.736 / -0.004
CrazyClimber	0.755 / 0.032	0.807 / 0.017	0.093 / -0.719	0.773 / -0.078
DemonAttack	0.633 / -0.041	0.391 / -0.255	0.453 / -0.147	0.740 / 0.002
Freeway	0.000 / -0.648	0.000 / -0.695	0.000 / -0.621	0.000 / -0.899
Frostbite	0.190 / -0.410	0.413 / -0.123	0.128 / -0.390	0.751 / 0.068
Gopher	0.846 / 0.245	0.789 / 0.093	0.410 / -0.270	0.912 / 0.138
Hero	0.643 / 0.229	0.248 / -0.330	0.011 / -0.522	0.772 / 0.157
Jamesbond	0.341 / -0.444	0.656 / -0.021	0.184 / -0.392	0.840 / 0.076
Kangaroo	0.892 / 0.317	0.883 / 0.262	0.468 / -0.148	0.865 / 0.136
Krull	0.611 / -0.090	0.308 / -0.434	0.165 / -0.490	0.449 / -0.372
KungFuMaster	0.596 / -0.071	0.493 / -0.196	-0.113 / -0.806	0.903 / 0.072
MsPacman	0.697 / 0.193	0.730 / 0.190	0.276 / -0.295	0.837 / 0.170
Pong	1.854 / 1.128	1.961 / 1.245	1.971 / 1.453	1.942 / 1.126
PrivateEye	7.146 / 5.984	1.423 / 0.613	-3.286 / -3.997	2.676 / 1.710
Qbert	0.854 / 0.176	0.966 / 0.505	-0.026 / -0.637	0.971 / 0.402
RoadRunner	0.887 / 0.396	0.550 / -0.106	0.275 / -0.384	0.607 / -0.104
Seaquest	0.275 / -0.295	0.241 / -0.370	0.023 / -0.550	0.332 / -0.515
UpNDown	0.709 / 0.196	0.676 / 0.129	0.233 / -0.285	0.890 / 0.084

Table 13: Per-task, per-attack-family results on DreamerV3 DMC. Each cell reports Drop / Utility for the best confirmed configuration within an attack family.

Task	APGD-CE	APGD-DLR	FAB	Square	PhysCond-WMA
acrobot swingup	0.997 / 0.180	0.986 / 0.235	0.991 / 0.178	0.201 / -0.655	0.872 / -0.134
ball in cup catch	0.235 / -0.280	0.411 / -0.132	0.865 / 0.264	0.115 / -0.447	0.817 / 0.010
cartpole balance	0.385 / -0.124	0.392 / -0.071	0.510 / 0.071	0.001 / -0.915	0.411 / -0.305
cartpole balance sparse	0.953 / 0.356	0.839 / 0.277	0.958 / 0.342	0.002 / -0.755	0.922 / 0.093
cartpole swingup	0.640 / 0.089	0.628 / 0.100	0.755 / 0.180	0.068 / -0.649	0.659 / -0.109
cartpole swingup sparse	0.988 / 0.455	0.974 / 0.389	0.994 / 0.362	0.028 / -0.643	0.960 / 0.159
cheetah run	0.535 / 0.083	0.575 / 0.141	0.845 / 0.445	0.386 / -0.152	0.843 / 0.147
finger spin	0.420 / -0.218	0.505 / -0.159	0.614 / -0.122	0.730 / -0.056	0.882 / 0.096
finger turn easy	0.233 / -0.176	0.276 / -0.124	0.553 / 0.110	0.178 / -0.279	0.339 / -0.372
finger turn hard	0.111 / -0.439	0.294 / -0.180	0.472 / 0.021	-0.124 / -0.771	0.343 / -0.385
hopper hop	0.799 / 0.342	0.831 / 0.373	0.950 / 0.590	0.224 / -0.491	0.971 / 0.289
hopper stand	0.948 / 0.150	0.921 / 0.407	0.993 / 0.468	-0.157 / -0.668	0.993 / 0.095
pendulum swingup	0.000 / -0.629	0.000 / -0.654	0.000 / -0.657	0.000 / -0.815	0.000 / -0.839
quadruped run	-0.022 / -0.380	-0.058 / -0.397	0.643 / 0.307	-0.172 / -0.550	0.925 / 0.228
quadruped walk	0.223 / -0.152	0.177 / -0.195	0.494 / 0.160	0.170 / -0.214	0.884 / 0.184
reacher easy	0.034 / -0.391	0.111 / -0.308	0.139 / -0.285	0.036 / -0.407	0.236 / -0.518
reacher hard	0.325 / -0.113	0.317 / -0.093	0.860 / 0.424	0.093 / -0.350	0.616 / -0.098
walker run	0.556 / 0.136	0.549 / 0.139	0.772 / 0.384	0.018 / -0.623	0.762 / 0.044
walker stand	0.068 / -0.394	-0.032 / -0.409	0.149 / -0.272	-0.043 / -0.506	0.192 / -0.532
walker walk	0.162 / -0.241	0.269 / -0.145	0.447 / 0.069	-0.013 / -0.475	0.321 / -0.385

F Efficiency-Curve Details

This section provides the numerical values behind the attack-specific efficiency curves in Figure 4. For each suite, attack family, and method, we report the mean best-so-far scalarized utility after 1, 2, 4, and 8 evaluated configurations. The averages are computed across tasks. The shaded regions in Figure 4 correspond to standard errors across tasks; here we report only the mean values for compactness.

Table 14: Attack-specific search efficiency on DreamerV3 Atari. Each value is the mean best-so-far scalarized utility after a given number of evaluated configurations.

Attack	Method	Trial 1	Trial 2	Trial 4	Trial 8
APGD-CE	Claudini	-0.175	-0.102	0.032	0.193
APGD-CE	Random	-0.161	-0.080	0.124	0.202
APGD-CE	WMAttack	0.183	0.522	0.582	0.651
APGD-DLR	Claudini	-0.158	-0.067	0.010	0.095
APGD-DLR	Random	-0.155	-0.114	-0.068	0.029
APGD-DLR	WMAttack	0.269	0.286	0.409	0.516
FAB	Claudini	-0.609	-0.401	-0.258	0.006
FAB	Random	-0.476	-0.438	-0.111	0.069
FAB	WMAttack	-0.355	-0.332	-0.236	-0.070
PhysCond-WMA	Claudini	-0.224	-0.182	0.171	0.259
PhysCond-WMA	Random	-0.260	-0.200	-0.099	0.010
PhysCond-WMA	WMAttack	0.796	0.801	0.925	1.015

Table 15: Attack-specific search efficiency on DreamerV3 DMC. Each value is the mean best-so-far scalarized utility after a given number of evaluated configurations.

Attack	Method	Trial 1	Trial 2	Trial 4	Trial 8
APGD-CE	Claudini	-0.303	-0.241	-0.196	-0.158
APGD-CE	Random	-0.273	-0.238	-0.175	-0.131
APGD-CE	WMAttack	-0.039	-0.011	0.024	0.037
APGD-DLR	Claudini	-0.365	-0.282	-0.196	-0.168
APGD-DLR	Random	-0.226	-0.206	-0.170	-0.151
APGD-DLR	WMAttack	-0.062	-0.045	0.005	0.065
Square	Claudini	-0.545	-0.476	-0.448	-0.400
Square	Random	-0.526	-0.472	-0.437	-0.375
Square	WMAttack	-0.621	-0.577	-0.515	-0.440
PhysCond-WMA	Claudini	-0.410	-0.377	-0.326	-0.275
PhysCond-WMA	Random	-0.384	-0.338	-0.320	-0.297
PhysCond-WMA	WMAttack	-0.106	-0.094	-0.069	-0.016

The Atari results show that WMAttack improves early-stage search quality for APGD-CE, APGD-DLR, and PhysCond-WMA, while FAB is more competitive. On DMC, WMAttack improves APGD-CE, APGD-DLR, and PhysCond-WMA, whereas Square Attack remains difficult for all methods and is better optimized by the baselines in this export. These results support the main observation that retrieval and feedback improve finite-budget search for most attack families, while the size of the gain depends on the attack family and task suite.

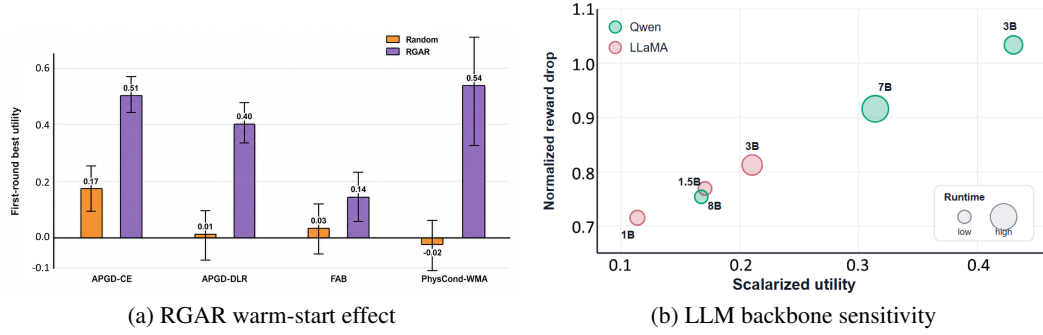


Figure 5: Design analysis of WMAttack on DreamerV3 Atari. RGAR improves first-round candidate quality, while LLM backbones induce different efficiency–accuracy trade-offs.

G Ablation Details

This section provides additional statistics for the RGAR ablation in Section 5.4. The experiment is conducted on DreamerV3 Atari with 26 tasks and four attack families: APGD-CE, APGD-DLR, FAB, and PhysCond-WMA. We compare RGAR initialization with random initialization while keeping the downstream SCAS protocol fixed. Each task–attack pair is treated as one comparison unit, resulting in 104 pairs.

Table 16: Per-attack RGAR ablation on DreamerV3 Atari. Each cell reports Utility / Drop / Flip averaged across 26 tasks.

Attack	Variant	First Utility / Drop / Flip $\uparrow$	Final Utility / Drop / Flip $\uparrow$
APGD-CE	Random init.	0.173 / 0.522 / 0.443	-0.243 / 0.395 / 0.439
APGD-CE	RGAR init.	0.505 / 0.813 / 0.658	0.206 / 0.795 / 0.650
APGD-DLR	Random init.	0.010 / 0.412 / 0.396	-0.354 / 0.322 / 0.393
APGD-DLR	RGAR init.	0.402 / 0.769 / 0.614	0.086 / 0.714 / 0.606
FAB	Random init.	0.032 / 0.363 / 0.227	-0.315 / 0.288 / 0.237
FAB	RGAR init.	0.140 / 0.483 / 0.255	-0.284 / 0.331 / 0.251
PhysCond-WMA	Random init.	-0.023 / 0.483 / 0.509	-0.389 / 0.393 / 0.514
PhysCond-WMA	RGAR init.	0.542 / 1.017 / 0.767	0.074 / 0.819 / 0.764

Table 17: Pair-level win counts for RGAR over random initialization on DreamerV3 Atari. Each entry reports the number of task–attack pairs where RGAR is strictly better than random initialization.

Metric	First round	Final result	Total pairs
Utility	85	87	104
Reward drop	84	85	104
Flip rate	96	92	104

These results show that RGAR improves both the initial proposal quality and the final search outcome for most task–attack pairs. The improvement is strongest for APGD-CE, APGD-DLR, and PhysCond-WMA, while FAB is more competitive. This is consistent with Figure 5(a), where RGAR provides a stronger warm start across all four attack families but the magnitude of improvement depends on the attack family.

Table 18: Per-attack final pair-level win counts for RGAR over random initialization. Each entry reports the number of tasks where RGAR achieves a higher final metric than random initialization.

Attack	Utility	Drop	Flip
APGD-CE	25 / 26	25 / 26	25 / 26
APGD-DLR	25 / 26	25 / 26	25 / 26
FAB	12 / 26	10 / 26	16 / 26
PhysCond-WMA	25 / 26	25 / 26	26 / 26